

GUÍA PRÁCTICA

Manejo de archivos en Python

Control de productos y órdenes de producción

Producto esperado: Un programa que trabaje durante toda la guía con el mismo archivo `datos/produccion.txt` y permita crear, agregar, leer, buscar, eliminar y actualizar registros.

1. Presentación de la actividad

En esta guía aprenderás el manejo básico y progresivo de archivos de texto en Python. No construirás cada ejercicio desde cero: comenzarás con un archivo vacío y, en cada etapa, agregarás una nueva funcionalidad al mismo programa.

El caso representa una pequeña empresa que necesita controlar productos y órdenes de producción. La situación puede analizarse desde dos perspectivas:

- Civil Industrial: producción, cantidades, costos, inventario y estado de las órdenes.
- Civil Informática: rutas, archivos, funciones, validación, búsqueda y actualización de datos.

Regla de trabajo: Todos los ejercicios deben utilizar el mismo archivo. No borres el archivo entre ejercicios, salvo que el docente lo indique para repetir una prueba.

2. Objetivos de aprendizaje

- Comprender qué es una ruta de archivo y cómo se utiliza en Python.
- Crear una carpeta y un archivo desde el programa.
- Diferenciar los modos de apertura `a`, `r` y `w`.
- Agregar información sin borrar registros anteriores.
- Leer y recorrer un archivo línea por línea.
- Separar los campos de un registro y convertir datos numéricos.
- Construir funciones pequeñas que puedan reutilizarse.
- Eliminar y actualizar registros reescribiendo el archivo de manera controlada.



UNIVERSIDAD
SAN SEBASTIAN



3. Caso y formato de los registros

La empresa necesita registrar órdenes de producción. Cada registro representará un producto que debe fabricarse o que ya fue procesado.

3.1. Ruta del archivo

```
datos/produccion.txt
```

La carpeta datos estará relacionada con la carpeta donde se ejecuta el programa. Si no existe, deberá crearse mediante código.

3.2. Formato de cada línea

```
id;producto;area;cantidad;valor_unitario;estado
```

Ejemplos de registros:

```
1;Silla;Muebles;10;25000;Pendiente  
2;Mesa;Muebles;5;45000;Completada
```

Importante: El punto y coma separa los campos. Por eso, no debe utilizarse dentro del nombre del producto, del área o del estado.

Campo	Nombre	Tipo	Ejemplo
1	id	int	1
2	producto	str	Silla
3	area	str	Muebles
4	cantidad	int	10
5	valor_unitario	int	25000
6	estado	str	Pendiente

3.3. Diccionario de ejemplo

```
producto = {  
    "id": 1,  
    "producto": "Silla",  
    "area": "Muebles",  
    "cantidad": 10,  
    "valor_unitario": 25000,  
    "estado": "Pendiente"  
}
```

El diccionario se utilizará dentro del programa. Para guardarlo en el TXT, sus valores deberán convertirse en una línea separada por punto y coma.

4. Modos de apertura que debes reconocer

Modo	Uso	Qué sucede
a	Agregar	Escribe al final y conserva lo anterior. Si no existe, crea el archivo.
r	Leer	Permite recorrer el contenido. El archivo debe existir.
w	Reescribir	Reemplaza el contenido. Se utilizará para eliminar o actualizar.
Seguridad básica: Utiliza siempre <code>with open(..., encoding="utf-8")</code> para que el archivo se cierre correctamente y se puedan guardar caracteres como á, é, í, ó y ú.		

5. Desarrollo progresivo de ejercicios

Realiza los siguientes ejercicios en orden. Después de cada ejercicio, ejecuta el programa y verifica el punto de control antes de continuar.

Ejercicio 1. Preparar la ruta y crear el archivo

Objetivo: Comprender que una ruta identifica la ubicación del archivo.

Instrucciones: Crea una constante para la carpeta datos y otra para el archivo produccion.txt. Usa pathlib.Path. Luego crea la carpeta con mkdir(...) y el archivo con modo a.

Pista: Puedes comenzar con: from pathlib import Path. La ruta puede construirse como Path("datos") / "produccion.txt".

Punto de control: Debe existir la carpeta datos y dentro de ella produccion.txt. El archivo puede estar vacío.

```
from pathlib import Path

CARPETA_DATOS = Path("datos")
ARCHIVO = CARPETA_DATOS / "produccion.txt"
```

Ejercicio 2. Agregar el primer registro

Objetivo: Escribir información al final de un archivo.

Instrucciones: Solicita al usuario el nombre de un producto y una cantidad. Guarda ambos datos en una línea. Por ahora puedes utilizar un formato sencillo, por ejemplo: producto;cantidad.

Pista: Abre el archivo con modo a y termina la línea con \n.

Punto de control: El archivo debe contener un registro y, al volver a ejecutar el programa, el registro anterior debe permanecer.

```
with open(ARCHIVO, "a", encoding="utf-8") as archivo:
    archivo.write(producto + ";" + str(cantidad) + "\n")
```

Ejercicio 3. Agregar varios registros

Objetivo: Reutilizar un bloque de instrucciones mediante un ciclo.

Instrucciones: Utiliza un ciclo para registrar tres productos. Valida que el nombre no esté vacío y que la cantidad sea un entero mayor que cero.

Pista: Usa `int()` dentro de `try/except` para controlar una entrada que no sea numérica.

Punto de control: El archivo debe conservar los registros anteriores y sumar tres nuevos registros.

Ejercicio 4. Leer y recorrer el archivo

Objetivo: Comprender cómo se recupera la información almacenada.

Instrucciones: Crea una función `leer_archivo()` que abra el archivo en modo `r` y recorra cada línea con un ciclo `for`. Muestra cada línea en pantalla.

Pista: `linea.strip()` elimina el salto de línea que viene desde el archivo.

Punto de control: Al ejecutar la función, cada registro debe aparecer en una línea limpia y ordenada.

```
def leer_archivo():  
    with open(ARCHIVO, "r", encoding="utf-8") as archivo:  
        for linea in archivo:  
            print(linea.strip())
```

Ejercicio 5. Separar los campos de cada registro

Objetivo: Convertir una línea de texto en datos que el programa pueda procesar.

Instrucciones: Usa `split(";")` para separar cada línea. Muestra por separado el producto y la cantidad. Convierte la cantidad a `int`.

Pista: Después de separar, los campos estarán en posiciones como `datos[0]` y `datos[1]`.

Punto de control: El programa debe mostrar una salida parecida a: Producto: Silla / Cantidad: 10.

Ejercicio 6. Completar el formato de producción

Objetivo: Trabajar con registros que contienen información de dos carreras.

Instrucciones: Amplía el registro para incluir `id`, `producto`, `area`, `cantidad`, `valor_unitario` y `estado`. Registra al menos cinco órdenes.

Pista: Mantén siempre el mismo orden de campos. Los montos y cantidades deben



guardarse como enteros.

Punto de control: Todas las líneas deben tener seis campos separados por punto y coma.

1;Silla;Muebles;10;25000;Pendiente



6. Procesar los registros con funciones

Ejercicio 7. Convertir cada línea en un diccionario

Objetivo: Reutilizar la información mediante una estructura clara.

Instrucciones: Crea una función `convertir_linea_a_diccionario(linea)`. La función debe separar la línea, convertir `id`, `cantidad` y `valor_unitario` a `int`, y retornar un diccionario.

Pista: Una función debe hacer una tarea concreta y retornar el resultado con `return`.

Punto de control: Al leer el archivo, debes obtener una lista de diccionarios, no solamente una lista de textos.

```
def convertir_linea_a_diccionario(linea):  
    datos = linea.strip().split(";")  
    producto = {  
        "id": int(datos[0]),  
        "producto": datos[1],  
        # completar los demás campos  
    }  
    return producto
```

Ejercicio 8. Mostrar los registros en formato legible

Objetivo: Separar la lectura de la presentación de los datos.

Instrucciones: Crea `mostrar_productos(productos)`. Recorre la lista de diccionarios y muestra `id`, `producto`, `área`, `cantidad`, `valor unitario` y `estado`.

Pista: Usa `enumerate()` si quieres numerar visualmente los registros.

Punto de control: La pantalla debe ser comprensible para una persona que no conoce el formato interno del TXT.

Ejercicio 9. Calcular costos de producción

Objetivo: Aplicar una operación útil para Civil Industrial.

Instrucciones: Para cada registro, calcula el costo estimado como `cantidad * valor_unitario`. Muestra el costo de cada orden y acumula un costo general.

Pista: No es necesario guardar todavía el costo calculado: puedes obtenerlo al leer la información.



Punto de control: El costo general debe coincidir con la suma de los costos individuales.

Ejercicio 10. Buscar por estado o área

Objetivo: Filtrar información usando condiciones.

Instrucciones: Solicita un estado o un área y muestra solamente los registros que coincidan. Prueba con Pendiente, Completada y Muebles.

Pista: La comparación puede realizarse con `if producto["estado"].lower() == busqueda.lower()`.

Punto de control: Los registros que no coinciden no deben aparecer en el resultado.

Ejercicio 11. Eliminar un registro

Objetivo: Actualizar un archivo de texto de forma controlada.

Instrucciones: Muestra los registros numerados. Solicita el número que se desea eliminar. Lee todos los registros, elimina uno de la lista y vuelve a escribir el archivo completo en modo w.

Pista: No intentes borrar una parte del archivo directamente. Primero trabaja con la lista y luego reescribe el archivo.

Punto de control: Después de eliminar, el registro no debe aparecer al volver a leer el archivo.

```
productos.pop(posicion - 1)

with open(ARCHIVO, "w", encoding="utf-8") as archivo:
    for producto in productos:
        # convertir el diccionario a una línea
        archivo.write(linea + "\n")
```

Ejercicio 12. Actualizar el estado de una orden

Objetivo: Modificar un registro existente.

Instrucciones: Permite seleccionar una orden y cambiar su estado, por ejemplo de Pendiente a Completada. Luego reescribe el archivo con la información actualizada.

Pista: La actualización se realiza en la lista de diccionarios antes de abrir el archivo en modo w.

Punto de control: Al cerrar y volver a ejecutar el programa, el nuevo estado debe mantenerse.

Ejercicio 13. Construir un menú

Objetivo: Integrar las funciones en un programa reutilizable.

Instrucciones: Crea un menú con opciones para preparar archivo, agregar, listar, buscar, eliminar, actualizar y salir. Cada opción debe llamar a una función distinta.

Pista: Utiliza `while True` y `break` para mantener el menú activo hasta elegir salir.

Punto de control: El programa debe permitir realizar varias acciones sin reiniciarse después de cada operación.

7. Funciones sugeridas para el programa final

Función	Retorna	Responsabilidad
<code>preparar_archivo()</code>	Nada	Crea carpeta y archivo si no existen.
<code>agregar_producto()</code>	Nada	Solicita datos y agrega una línea.
<code>leer_productos()</code>	Lista	Lee y reconstruye diccionarios.
<code>mostrar_productos(lista)</code>	Nada	Presenta los registros.
<code>buscar_productos(lista)</code>	Lista	Filtra por área o estado.
<code>eliminar_producto(lista)</code>	Nada	Elimina y reescribe el archivo.
<code>actualizar_estado(lista)</code>	Nada	Modifica y reescribe el archivo.
<code>programa_principal()</code>	Nada	Coordina el menú.

8. Pruebas que debes realizar

- Ejecutar el programa por primera vez y verificar que cree `datos/produccion.txt`.
- Cerrar el programa, ejecutarlo nuevamente y confirmar que los registros anteriores permanezcan.
- Agregar una entrada vacía y comprobar que sea rechazada.
- Ingresar una cantidad con letras y comprobar que el programa no se cierre.
- Buscar una categoría que exista y otra que no exista.



- Eliminar una orden y comprobar que desaparezca del archivo.
- Actualizar el estado de una orden y volver a leer el archivo.
- Revisar manualmente el contenido de datos/produccion.txt.

9. Entrega y producto final

Entrega una carpeta que contenga el programa y el archivo generado. El archivo TXT debe haber sido creado por el programa, no creado manualmente antes de ejecutarlo.

- programa_produccion.py, con funciones y comentarios.
- datos/produccion.txt generado durante las pruebas.
- Capturas de la ejecución de las operaciones principales.
- Una breve reflexión: ¿qué diferencia observaste entre los modos a, r y w?

Criterio de logro: El programa mantiene información en un archivo de texto, permite trabajar con ella mediante funciones y conserva los cambios entre ejecuciones.

10. Preguntas de reflexión

1. ¿Qué ocurre si se utiliza w en vez de a para agregar un registro?
2. ¿Por qué es necesario usar strip() al leer una línea?
3. ¿Por qué cantidad y valor_unitario deben convertirse a int?
4. ¿Qué ventaja tiene trabajar con diccionarios en vez de usar posiciones como datos[0], datos[1] y datos[2] en todo el programa?
5. ¿Qué problema podría ocurrir si dos registros tienen el mismo id?